

API подписки/отписки

API подписки на событие

Подписаться на событие можно в сервисах БЛ и клиентских приложениях.

Чтобы подписаться на событие из исходного кода в C++, используйте методы класса `event::Subscription`:

 [API подписки на событие для мобильных и десктоп приложений](#)

API C++ класса `event::Subscription` для подписки на события.

Название	Описание
<code>SetChannel</code>	Устанавливает тип прослушиваемого канала/события. Имеет один строковый параметр.
<code>SetUserLogin</code>	Устанавливает логин служебного пользователя, под которым будет запущен callback обработчика события на сервисе БЛ. Имеет один строковый параметр.
<code>SetCallback</code>	Устанавливает callback обработчика события. Параметр — имя метода БЛ. Этот метод шаблонный и принимает n параметров: - Первый параметр – строка с именем метода БЛ. - Остальные параметры — соответствуют сигнатуре метода БЛ, используемого в качестве обработчика. Если для обработчика события требуется payload, то используется специальный маркер.
<code>SetIsForInstance</code>	Подписка для конкретного экземпляра или для группы серверов. Параметры: <code>true</code>. Подписка для конкретного экземпляра. Событие попадет в каждую подписавшуюся БЛ. <code>false</code>. Подписка для группы серверов. Событие попадет на какую-то произвольную БЛ подписавшейся группы серверов. Значение по умолчанию. Если установлено значение <code>True</code> , то используются параметры "Хост" и "Порт" базовой конфигурации. "Хост" и "Порт" приходят вместе с остальной конфигурацией из управления облаком. При необходимости, их можно переопределить локально в ini-файле.
<code>SetIsDurable</code>	Устанавливает гарантированную доставку сообщений. При работе в режиме с гарантированной доставкой не полученные группой серверов сообщения хранятся на Брокере. Для всех подписок по умолчанию выбрана доставка сообщений без гарантии. Параметры: <code>true</code>. Гарантированная доставка сообщений. <code>false</code>. Доставка сообщений без гарантии.
<code>Add</code>	Добавляет обработчик на событие.
<code>Delete</code>	Удаляет обработчик.

API Python класса `event.Subscription` для подписки на события.

Атрибут/Метод	Описание
channel	Атрибут. Установка типа прослушиваемого канала/события.
user_login	Атрибут. Установка логина служебного пользователя, под которым будет запущен callback обработчика события на сервисе БЛ.
SetCallback	Метод. Устанавливает callback обработчика события. Аргумент метода — имя метода БЛ. Этот метод шаблонный и принимает n параметров: - Первый параметр – строка с именем метода БЛ. - Остальные параметры — соответствуют сигнатуре метода БЛ, используемого в качестве обработчика. Если для обработчика события требуется payload, то используется специальный маркер.
for_instance	Атрибут. Подписка для конкретного экземпляра или для группы серверов. Значения: - True. Подписка для конкретного экземпляра. Событие попадет в каждую подписавшуюся БЛ. - False. Подписка для группы серверов. Событие попадет на какую-то произвольную БЛ подписавшейся группы серверов. Значение по умолчанию. Если установлено значение True, то используются параметры "Хост" и "Порт" базовой конфигурации. "Хост" и "Порт" приходят вместе с остальной конфигурацией из управления облаком. При необходимости, их можно переопределить локально в ini-файле.
durable	Атрибут. Установка гарантированной доставки сообщений. При работе в режиме с гарантированной доставкой не полученные группой серверов сообщения хранятся на Брокере. Для всех подписок по умолчанию выбрана доставка сообщений без гарантии. Параметры: true (Гарантированная доставка сообщений) и false (Доставка сообщений без гарантии).
Add	Метод. Добавляет обработчик на событие.
Delete	Метод. Удаляет обработчик.

JavaScript API для подписки на события.

Метод	Описание
serverChannel	Задаёт канал, в который публикуется событие. Единственный параметр — имя канала. Возвращает объект channel.
subscribe	Добавляет обработчик на событие. Имеет три параметра: - Строка с именем внутреннего события платформы, которое будет ассоциировано с каналом сервисных событий. - Функция-обработчик с двумя параметрами event и data. В event приходит payload события, разобранный функцией JSON.parse(). В data приходит frame WebSocket-сообщения. - Позволяет задать контекст, в котором будет вызван обработчик. Необязательный параметр.

Ограничения

Если для обработчика события в качестве одного из параметров используется объект события (payload), то при несовпадении типа объекта события и параметра в сигнатуре обработчика возникнет исключение при сериализации.

Требования к программному окружению

Для подписки из исходного кода C++:

1. В исходном файле подключите заголовочный файл sbis-event-subscription/subscription.hpp.
2. В состав сервиса приложения включите сервисный модуль EventSubscription.
3. В свойствах вашего сервисного модуля добавьте зависимость от сервисного модуля EventSubscription.
4. В пользовательской библиотеке в файле CMakeLists.txt добавьте sbis_add_sdk_dependency("sbis-event-subscription300").

Для подписки из исходного кода Python:

1. В исходном файле подключите модуль event.
2. В состав сервиса приложения включите сервисные модули EventSubscription и EventSubscription-Py.
3. В свойствах вашего сервисного модуля добавьте зависимость от сервисных модулей EventSubscription и EventSubscription-Py.

Для подписки из исходного кода JavaScript необходимо импортировать класс Browser/Event:Server.

Примеры подписки из Python и C++



Примеры для подписки из мобильного и десктопного приложения описаны [здесь](#).

Примеры для сервиса в облаке.

1. Чтобы установить для события обработчик, не имеющий параметров, выполните следующее:

- C++

```
1 class YOUR_DLL_FUNC ModuleInitializerHandle : public
  AbstractModuleHandle
2 {
3 public:
4     bool OnEndAllLoadModules( EndLoadEventAnswer& ) const override
5     {
6         event::Subscription s;
7         s.SetChannel( L"fed.specificationhasbeenupdated" );
8         s.SetUserLogin( L"online_service" );
9         s.SetCallback(
10         L"ТипФормализованногоДокумента.ОчиститьКэшСтенда" );
11         s.Add();
12         return True;
13     };
14 }
```

- Python

```
1 def OnEndAllLoadModules(answer):
2     import event
3     s = event.Subscription()
4     s.channel = 'fed.specificationhasbeenupdated'
5     s.user_login = 'online_service'
6     s.SetCallback( 'ТипФормализованногоДокумента.ОчиститьКэшСтенда'
7 )
8     s.Add()
```

2. Чтобы установить для события обработчик, принимающий **один** параметр, и передать обработчику события в качестве параметра payload, используйте маркер event::_MsgBody_ или event._MsgBody_ следующим образом:

- C++

```

1 class YOUR_DLL_FUNC ModuleInitializerHandle : public
  AbstractModuleHandle
2 {
3 public:
4     bool OnEndLoadAllModules( EndLoadEventAnswer& ) const override
5     {
6         event::Subscription s;
7         s.SetChannel( L"fed.specificationhasbeenupdated" );
8         s.SetUserLogin( L"online_service" );
9         s.SetCallback(
10            L"ТипФормализованногоДокумента.ОчиститьКэшСтенда",
11            sbis::rpc::_MsgBody_ );
12         s.Add();
13         return True;
14     }
15 };

```

- Python

```

1 def OnEndAllLoadModules(answer):
2     import event
3     s = event.Subscription()
4     s.channel = 'fed.specificationhasbeenupdated'
5     s.user_login = 'online_service'
6     s.SetCallback(
7         'ТипФормализованногоДокумента.ОчиститьКэшСтенда', event._MsgBody_ )
8     s.Add()

```

Здесь payload будет передан обработчику первым параметром.

3. Чтобы установить для события обработчик, принимающий **несколько** параметров, и передать ему в качестве одного из параметров payload, используйте маркер sbis::rpc::_MsgBody_ или event._MsgBody_ следующим образом:

- C++

```

1 class YOUR_DLL_FUNC ModuleInitializerHandle : public
  AbstractModuleHandle
2 {
3 public:
4     bool OnEndLoadAllModules( EndLoadEventAnswer& ) const override
5     {
6         event::Subscription s;
7         s.SetChannel( L"fed.specificationhasbeenupdated" );
8         s.SetUserLogin( L"online_service" );
9         s.SetCallback(
10            L"ТипФормализованногоДокумента.ОчиститьКэшСтенда", "",
11            sbis::rpc::_MsgBody_, param1, param2 );
12         s.Add();
13         return True;
14     }
15 };

```

- Python

```

1 def OnEndAllLoadModules(answer):
2     import event
3     s = event.Subscription()
4     s.channel = 'fed.specificationhasbeenupdated'
5     s.user_login = 'online_service'
6     s.SetCallback(

```

```

7   'ТипФормализованногоДокумента.ОчиститьКэшСтенда', '',
    event._MsgBody_, param1, param2 )
    s.Add()

```

Здесь payload будет передан обработчику вторым параметром. Остальные параметры будут иметь фиксированное значение, установленное в методе SetCallback().

4. Чтобы установить обработчик для единичного экземпляра сервиса БЛ, используйте метод SetIsForInstance() или атрибут for_instance:

- C++

```

1  class YOUR_DLL_FUNC ModuleInitializerHandle : public
    AbstractModuleHandle
2  {
3  public:
4      bool OnEndLoadModules( EndLoadEventAnswer& ) const override
5      {
6          event::Subscription s;
7          s.SetChannel( L"fed.specificationhasbeenupdated" );
8          s.SetUserLogin( L"online_service" );
9          s.SetCallback(
    L"ТипФормализованногоДокумента.ОчиститьКэшСтенда" );
10         s.SetIsForInstance(true);
11         s.Add();
12         return True;
13     }
14 };

```

- Python

```

1  def OnEndAllLoadModules(answer):
2      import event
3      s = event.Subscription()
4      s.channel = 'fed.specificationhasbeenupdated'
5      s.for_instance = True
6      s.user_login = 'online_service'
7      s.SetCallback( 'ТипФормализованногоДокумента.ОчиститьКэшСтенда'
8      )
9      s.Add()

```

5. Чтобы установить гарантированную доставку сообщения события, используйте метод SetIsDurable() или атрибут durable:

- C++

```

1  class YOUR_DLL_FUNC ModuleInitializerHandle : public
    AbstractModuleHandle
2  {
3  public:
4      bool OnEndLoadModules( EndLoadEventAnswer& ) const override
5      {
6          event::Subscription s;
7          s.SetChannel( L"fed.specificationhasbeenupdated" );
8          s.SetUserLogin( L"online_service" );
9          s.SetCallback(
    L"ТипФормализованногоДокумента.ОчиститьКэшСтенда" );
10         s.SetIsDurable(true);
11         s.Add();
12         return True;
13     }
14 };

```

- Python

```

1 def OnEndAllLoadModules(answer):
2     import event
3     s = event.Subscription()
4     s.channel = 'fed.specificationhasbeenupdated'
5     s.durable = True
6     s.user_login = 'online_service'
7     s.SetCallback( 'ТипФормализованногоДокумента.ОчиститьКэшСтенда'
8 )
9     s.Add()

```

Примеры подписки из JavaScript

Пример 1. Если производится подписка на событие, генерируемое ресурсоёмким запросом, то сначала нужно дождаться готовности канала (событие `onReady`), а затем — события `onMessage`.

```

1 import { Server } from 'Browser/Event';
2 const channel = Server.serverChannel('MyBLEvent');
3 channel.once('onready', () => {
4     channel.subscribe('onmessage', () => {...});
5
6     // Соединение с сервером установлено, можно вызывать БЛ.
7     // Если вызвать до onready, то есть вероятность, что БЛ опубликует
    событие до того,
8     // как инициализируется подключение к инфраструктуре серверных
    событий.
9     sbisService.call('generateMyBLEvent');
10 });

```

Пример 2. Подписка на канализированное событие.

Для подписки на канализированное событие из JavaScript применяется метод `serverChannel()`. Вторым аргументом нужно передать подопцию `isChanneled` со значением `true`.

```

1 import { Server } from 'Browser/Event';
2
3 // Доступ как к функции - создание канала событий.
4 const channel = Server.serverChannel('my-channel', { isChanneled: true
5 });
6
7 // Доступ как к объекту
8 Server.serverChannel.close('my-channel'); // Закрыть серверный канал,
    чтоб данные больше не приходили в браузер.

```

API для отписки от событий в клиентских приложениях



[Отписка от события для мобильных и десктоп приложений](#)

Для отписки от событий используйте методы.

API JavaScript для отписки от событий.

Метод	Описание
unsubscribe	Удаление одного обработчика с события.

```

1  define('MyModule/MyComponent/J
   SModuleName',
2  [..., 'Browser/Event'],
3  function(..., bus) {
4
5      // В модуле должна быть
   объявлена именованная функция.
6      // На неё производим
   подписку или отписку.
7      function MyHandler (event,
   data) {
8          // event – объект
   события (payload).
9          // data – фрейм
   WebSocket-сообщения.
10         if (data ===
   'Поощрение') {
11             updatePromotion();
12         }
13         if (data ===
   'Взыскание') {
14             updatePenalty();
15         }
16     }
17     var moduleClass =
   CompoundControl.extend({
18         ...
19         var channel =
   bus.Server.serverChannel('insi
   de.informerhasbeenclicked');
20         ...
21
22         // Отписываемся от
   события.
23         // onMessage – имя
   внутреннего события веб-
   фреймворка WS,
24         // которое
   ассоциируется с каналом
   сервисных событий.
25
   channel.unsubscribe('onMessage
   ', MyHandler);
26         ...
27     });
28     return moduleClass;
29 });

```

[unbind](#)

Удаление всех обработчиков на события.

```

1  require(['Browser/Event'],
   function(bus) {
2      // onMessage – имя
   внутреннего события веб-
   фреймворка WS,
3      // которое ассоциируется с

```

каналом сервисных событий.

4

```
bus.Server.serverChannel('inside.informerhasbeenclicked').un  
bind('onMessage');
```

5

```
});
```